Lab 1 - Building a CI/CD Pipeline with GitHub Actions

Date: 23 October 2026

Repository: https://github.com/salxz696969/ci-cd-lab

Source assignment: `Lab 1 - CI_CD.pdf` (seven pages, copied unchanged into `docs/`).

Objective and terminology

The objective is to build a small web application, verify it locally, and make GitHub repeat the checks automatically for changes to the repository.

Continuous integration (CI) means checking changes frequently with an automated pipeline. Continuous delivery means keeping a tested release ready to deploy, with a human usually deciding when to release. Continuous deployment goes one step further and automatically releases a change after all required checks pass. This assignment implements CI. Uploading a coverage report is publishing evidence, not deploying a running application.

Steps 1-2: Project and package configuration

The `ci-cd-lab` folder was created from scratch with the handout's source files, `test/` and `.github/workflows/`. No starter archive was used. `package.json` declares `type: module`, so `.js` files use `import` and `export`. It requires Node.js 24 or later and pins the package manager to Yarn 4.18.1.

`express` is the application dependency. Mocha, Chai, Supertest, ESLint and c8 are development dependencies used to validate it. The scripts are:

Command	Purpose
`yarn start`	Start `server.js`.
`yarn lint`	Run ESLint over JavaScript files.
`yarn test`	Run the Mocha test suite.
`yarn coverage`	Run Mocha through c8 and write coverage reports.

The laptop's default Node.js is 22.23.3. A separate Node.js 24.22.1 executable was downloaded from the official Node.js distribution and its SHA-256 hash was checked against the published checksum. The existing default installation was not replaced. Final verification uses Node.js 24.22.1 and Yarn 4.18.1.

Steps 3-4: Application and server

`app.js` creates an Express application. Its `GET /` handler sends status 200 and the exact text `Hello, GitHub Actions!`. Exporting the app lets tests import the same application used by the server.

`server.js` imports that app and starts its HTTP listener. The expression `process.env.PORT ?? 3000` uses the supplied port, or 3000 when the value is null or undefined. Separating routing from startup lets tests exercise the application without opening the development server's fixed port.

Step 5: Automated HTTP tests

Mocha's `describe` groups the tests and `it` defines each case. Chai's `expect` compares actual and expected values. Supertest sends a request to the imported Express app, with a temporary listener managed by the test library.

The baseline has two tests: one asserts HTTP 200 and one asserts the exact greeting. Both passed under Node.js 24.22.1. The status assertion alone would not detect incorrect response text, which is why both checks matter.

Steps 6-7: Lint and dependency installation

`eslint.config.js` uses ESLint's flat configuration and recommended JavaScript rules. Node.js globals allow `process` and `console`; Mocha globals allow `describe` and `it`. Generated coverage, installed dependencies and Yarn's generated files are ignored.

`.yarnrc.yml` selects `nodeLinker: node-modules`. This file is listed in the handout's project structure but its content is not supplied, so the explicit setting completes that requirement. Yarn generated `yarn.lock`, which is committed. The immutable install passed: CI can reproduce the dependency resolution without silently editing that lockfile.

```
corepack enable
corepack prepare yarn@4.18.1 --activate
yarn install --immutable
```

The dependency ranges from the handout were retained. The lockfile records the actual resolved packages, including Express 5.3.0, ESLint 10.13.0, Mocha 12.0.3, Chai 6.3.0, Supertest 7.3.1 and c8 10.1.3.

Steps 8-9: Local server and deliberate failure

Port 3000 already belongs to another development service, so this lab server uses `PORT=3001`. An HTTP/browser check of http://localhost:3001/ returned the expected greeting. This demonstrates the port override without interrupting the existing service. The server is left running for further inspection.

Lint passed with exit code 0. The baseline tests passed (two passing, zero failing). For the requested experiment, the response was temporarily changed to `Hello, broken greeting!`. The status test still passed but the response test failed: one passing, one failing, exit code 1. The original greeting was restored and the tests passed again. The broken version was never committed. The captured assertion is in `evidence/intentional-failure.txt`.

Step 10: Coverage and interpretation

c8 uses V8 execution data to estimate which source code the tests exercised. Statement coverage measures executed statements; branch coverage measures execution paths; function coverage measures executed functions; line coverage measures executed source lines. Results depend on how V8/c8 map the code.

Both `app.js` and `server.js` are explicitly included, with `all: true`, so an unimported source file cannot silently disappear from the report. Reporters write the terminal table, `coverage/index.html`, and `coverage-summary.json`.

Baseline results under Node.js 24.22.1:

File	Statements	Branches	Functions	Lines
app.js	100%	100%	100%	100%
server.js	0%	0%	0%	0%
All files	56.25%	66.66%	0%	56.25%

The tests import `app.js`, so they do not exercise `server.js`. The total function metric is 0% because c8 counted no functions in the covered app file and one function in the unexecuted server file. The app's displayed 100% for functions is consequently vacuous, not proof of thorough function testing. The browser check of the separately running server does not contribute to Mocha's coverage process. A future improvement is an automated subprocess test for startup and the default/custom port paths.

The optional threshold extension is not enabled. `yarn coverage` generates the report and fails if tests fail, but low coverage alone does not fail CI. Even 100% coverage would not prove correctness: untested requirements, error cases and assertions can still be missing.

Step 11: GitHub Actions workflow

`.github/workflows/ci.yml` runs on pushes to `main`, pull requests targeting `main`, and manual dispatch. Its `quality` job runs on `ubuntu-latest`.

1. Check out the repository with `actions/checkout@v4`.

2. Enable Corepack and activate Yarn 4.18.1.

3. Set up Node.js 24 with `actions/setup-node@v4` and Yarn caching.

4. Install dependencies with `yarn install --immutable`.

5. Run lint, tests, then coverage.

6. Upload the `coverage/` directory with `actions/upload-artifact@v4` as `code-coverage-report`, even when an earlier step fails if a report exists.

The explicit Yarn activation repairs a gap in the supplied example: cache discovery can invoke Yarn before the install step. Activation must occur before `setup-node` looks up the Yarn cache. The package-manager version in the workflow matches `package.json`. Read-only repository permissions suffice for this workflow. No credentials or deployment secrets are stored in source.

Steps 12-14: Remote evidence and pull-request exercise

The private repository is https://github.com/salxz696969/ci-cd-lab. The initial implementation was committed on `main` as `1a68e10`. Its push triggered run 39917635906, which completed successfully. Installation, lint, both tests, coverage generation and artifact upload all passed. The downloaded `code-coverage-report` artifact was inspected; its summary matched the local baseline measurements.

Main-branch run: https://github.com/salxz696969/ci-cd-lab/actions/runs/39917635906

After checking the working tree and fetching `origin`, `feature/health-route` was created from the current `origin/main`. The feature adds `GET /health`, which returns HTTP 200 and JSON `{ "status": "ok" }`. Its additional test checks the status, JSON content type and exact response object. Local lint, all three tests and coverage passed before the branch was pushed.

Pull request 1 targets `main` and is left open for review: https://github.com/salxz696969/ci-cd-lab/pull/1

The pull-request workflow for code commit `210c413` also completed successfully. Run: https://github.com/salxz696969/ci-cd-lab/actions/runs/39917911066

Final local and hosted coverage for the health-route version:

File	Statements	Branches	Functions	Lines
app.js	100%	100%	100%	100%
server.js	0%	0%	0%	0%
All files	65%	75%	0%	65%

The health route adds covered lines, so total line coverage rises from 56.25% to 65%; the startup coverage gap remains. Three tests pass in the final suite. Both hosted artifacts were downloaded and their HTML/JSON reports inspected. The `evidence/` folder contains the local output, deliberate-failure assertion, main and pull-request run details and logs, authenticated GitHub screenshots, and local app and coverage screenshots.

GitHub reports a non-fatal warning that the handout's v4 actions target the deprecated Node.js 20 action runtime and are being forced to run under Node.js

24. It also reports the `ubuntu-latest` image migration. The successful runs verify current execution; a maintained project should update these action versions and validate the update. The application's configured runtime is Node.js 24, independent of the action implementation runtime.

Because the repository is private, an instructor needs repository access to open the live run links. No merge into `main` was performed: the open pull request preserves the lab's review exercise.

Reflection

The deliberate greeting error showed that passing HTTP status checks alone cannot establish correct behavior. CI makes both checks run consistently for future changes. The lockfile and pinned Yarn version reduce differences between local and hosted installs. Separating app construction from listening made HTTP tests straightforward, while whole-project coverage made the remaining server-startup gap visible. The next useful improvements would be startup tests, explicit error responses, and coverage thresholds chosen only after those behaviors are meaningfully tested.

References

- Supplied assignment: `docs/Lab 1 - CI_CD.pdf`, steps 1-14.

- Yarn Corepack documentation: https://yarnpkg.com/corepack

- GitHub's setup-node action: https://github.com/actions/setup-node

- Node.js 24 distribution and checksums: https://nodejs.org/dist/latest-v24.x/

Appendix A - Final source code

The repository contains the full project and lockfile. These files show the core application and HTTP tests.

app.js

```
import express from 'express';

const app = express();

app.get('/', (req, res) => {
  res.status(200).send('Hello, GitHub Actions!');
});

app.get('/health', (req, res) => {
  res.status(200).json({ status: 'ok' });
});

export default app;
```

server.js

```
import app from './app.js';

const port = process.env.PORT ?? 3000;

app.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});
```

test/test.js

```
import { expect } from 'chai';
import request from 'supertest';
import app from '../app.js';

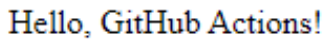
describe('GET /', () => {
  it('returns HTTP 200', async () => {
    const response = await request(app).get('/');
    expect(response.status).to.equal(200);
  });

  it('returns the expected greeting', async () => {
    const response = await request(app).get('/');
    expect(response.text).to.equal('Hello, GitHub Actions!');
  });
});

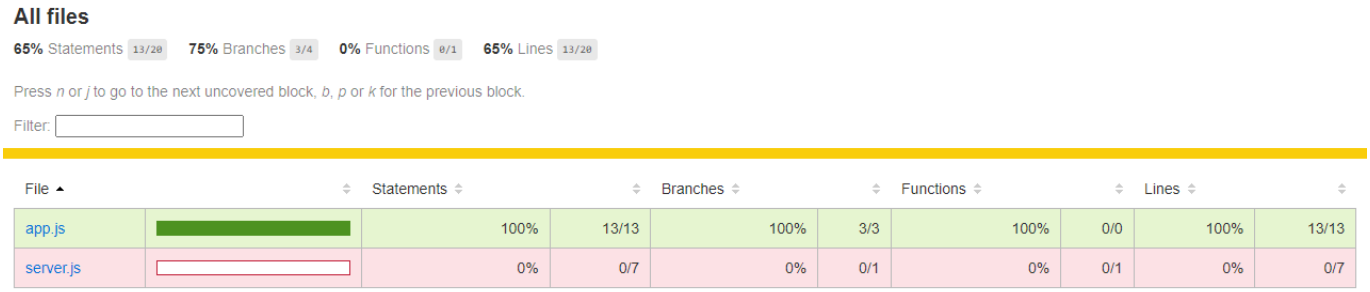
describe('GET /health', () => {
  it('returns HTTP 200 and a JSON health status', async () => {
    const response = await request(app).get('/health');
    expect(response.status).to.equal(200);
    expect(response.headers['content-type']).to.match(/application\/json/);
    expect(response.body).to.deep.equal({ status: 'ok' });
  });
});
```

Appendix B - Local verification screenshots

Local root endpoint: the browser displays the required greeting.



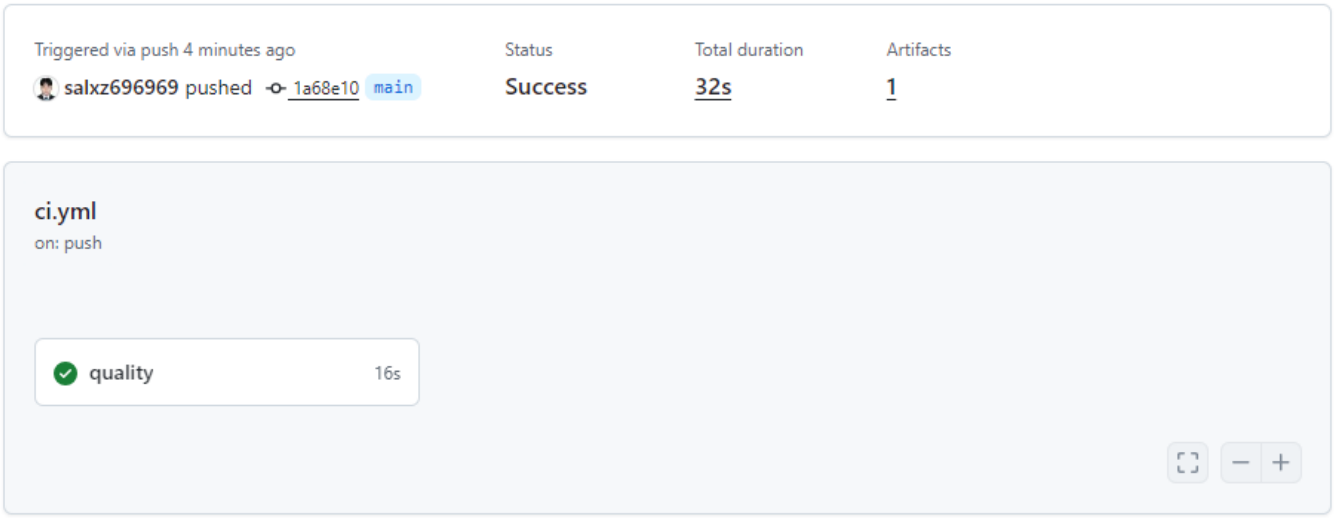
Final local coverage: app.js is covered; server.js remains outside the automated HTTP tests.



Appendix C - Successful GitHub workflows

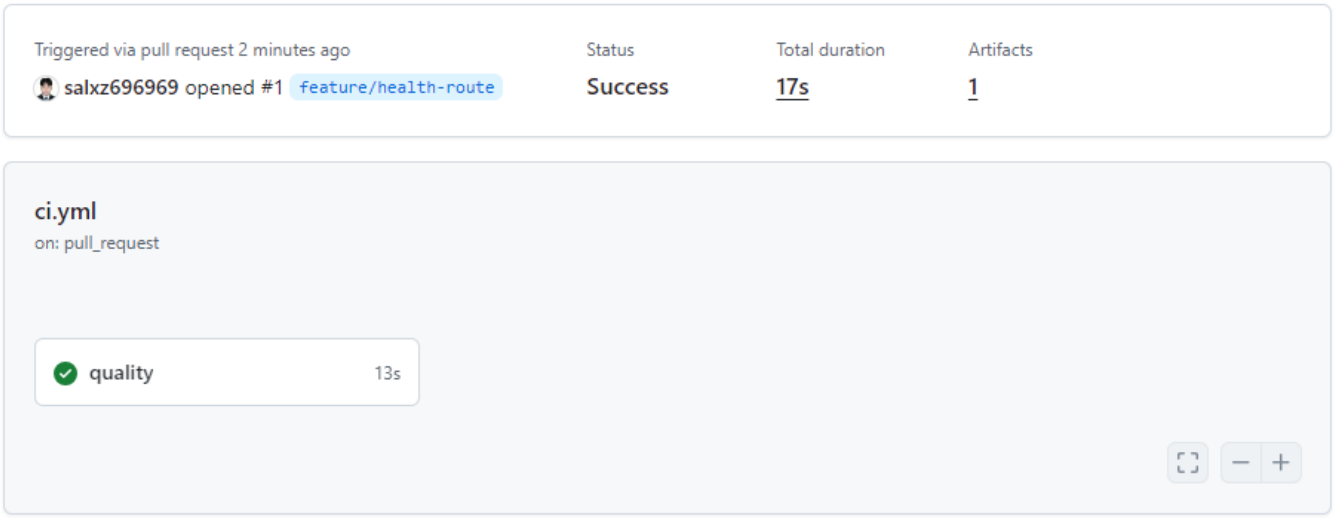
Cropped workflow summaries from the authenticated GitHub pages. Full screenshots and run logs are included in evidence/.

Successful main-branch workflow



Verified run: https://github.com/salxz696969/ci-cd-lab/actions/runs/39917635906

Successful pull-request workflow



Verified run: https://github.com/salxz696969/ci-cd-lab/actions/runs/39917911066